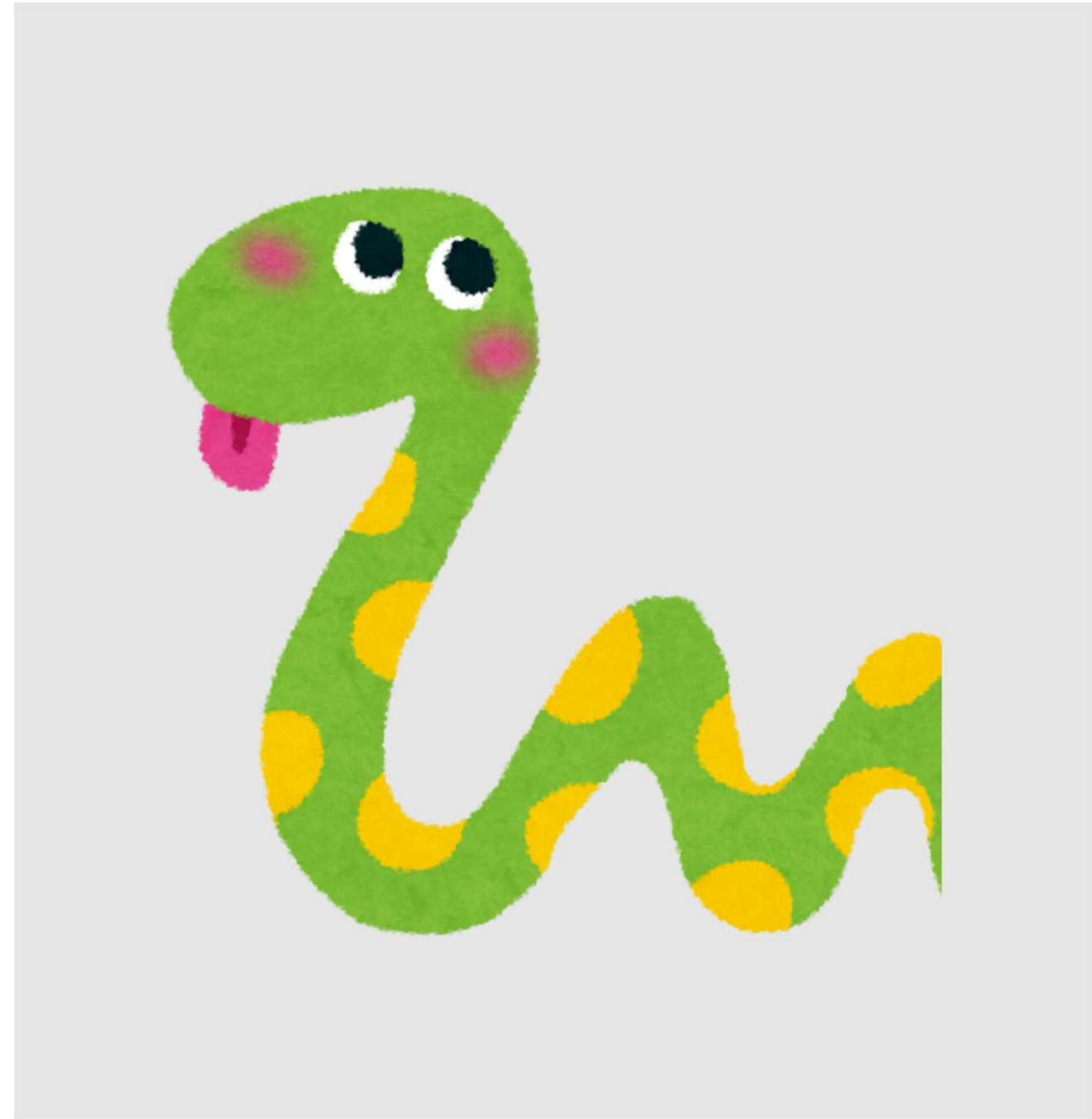
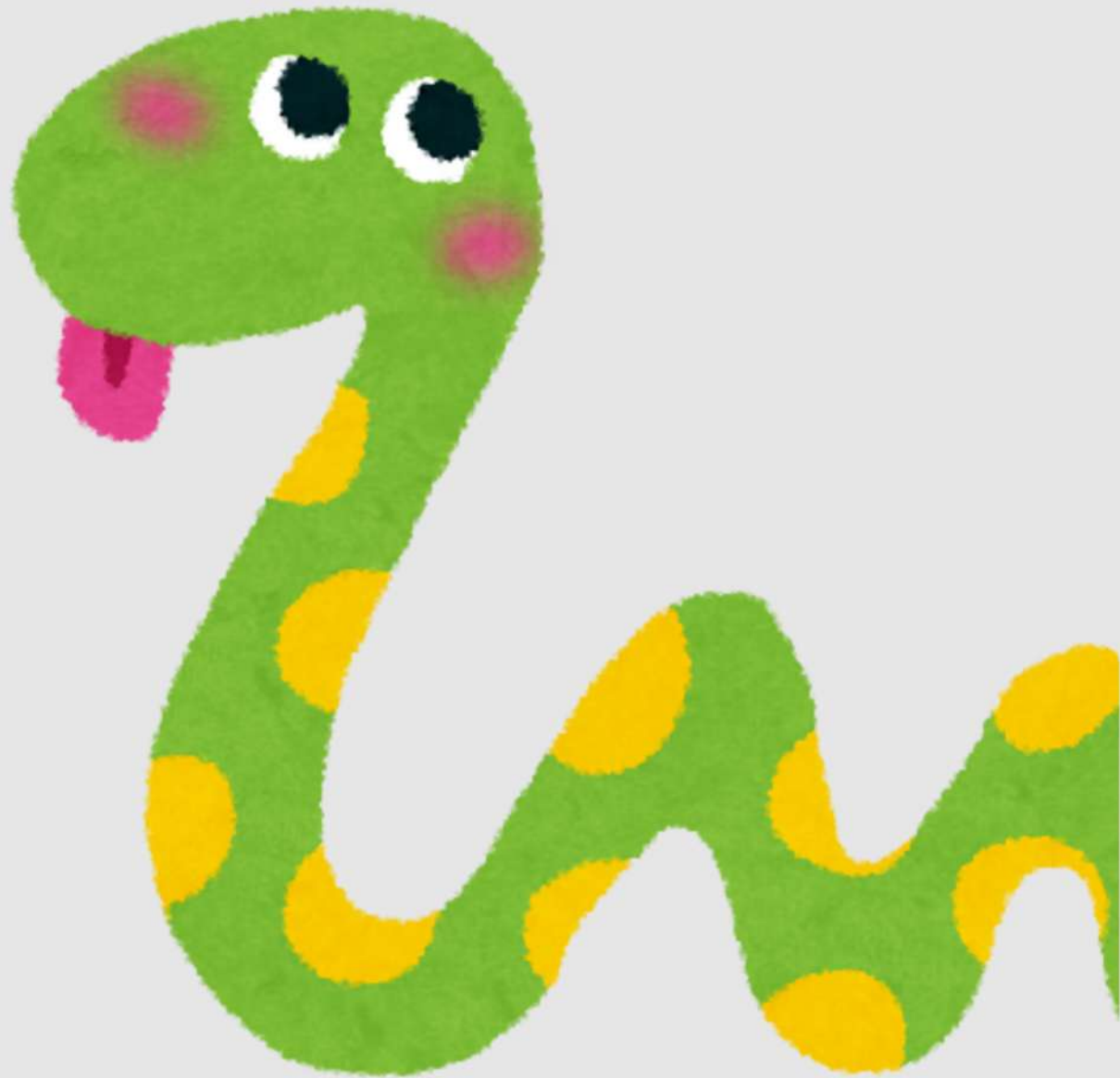


Jeu du serpent (Snake) en langage C

PROJET DE PROGRAMMATION – BTS
2^e ANNÉE





Objectif

CONCEVOIR ET PROGRAMMER EN C UN JEU SNAKE

L'objectif du projet est de créer un jeu du serpent (Snake) en langage C.

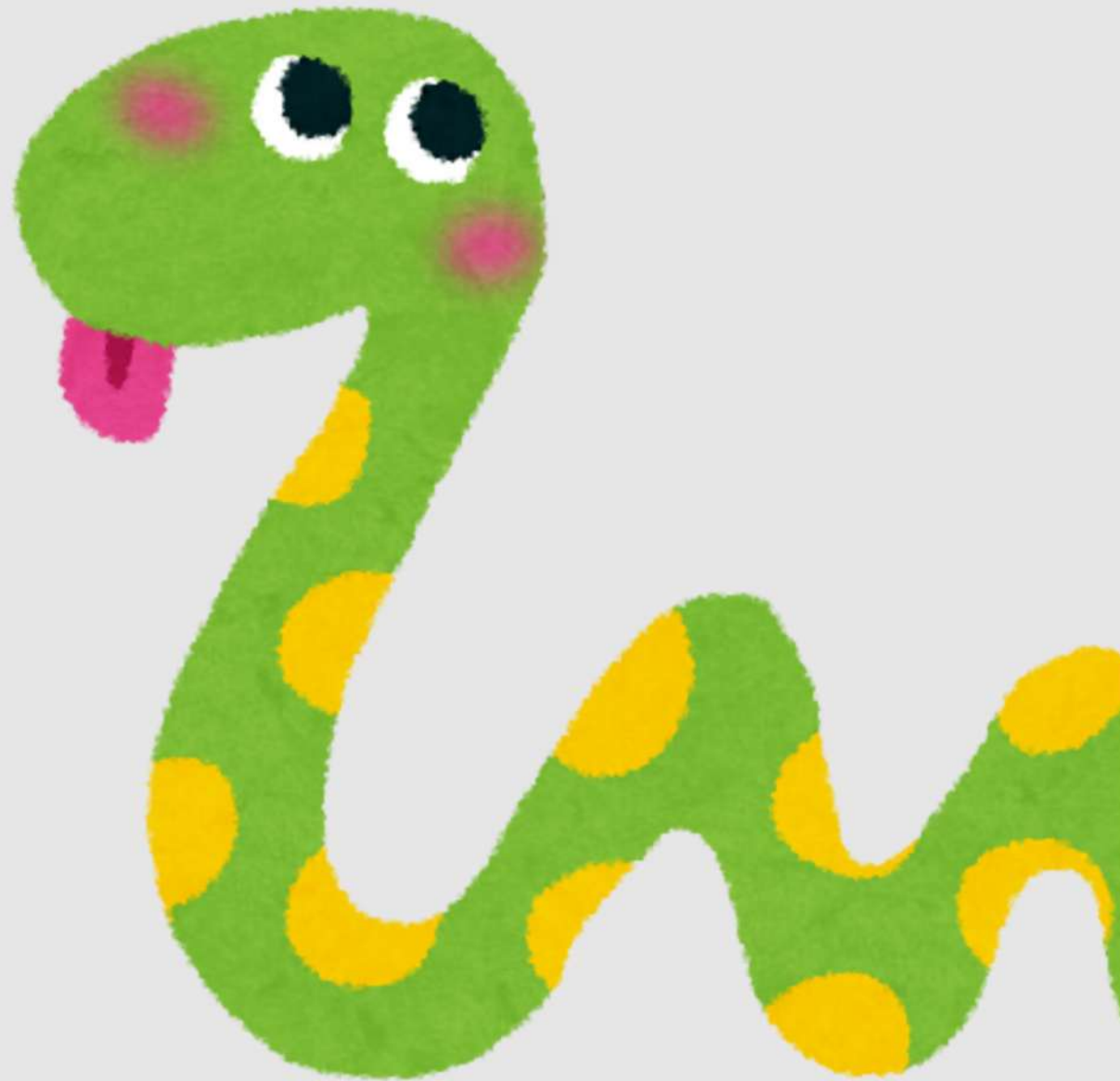
Le joueur contrôle un serpent qui se déplace dans une grille à l'aide du clavier.

Le but est de manger les pommes qui apparaissent aléatoirement à l'écran pour marquer des points.

À chaque fois que le serpent mange trois pommes, il grandit.

La partie se termine si le serpent touche un mur ou sa propre queue.

Ce projet permet de mettre en pratique la programmation structurée, la gestion des tableaux, fichiers et entrées clavier en C.



Présentation du groupe



- Syd BELLO (Responsable technique)
- Aymeric TROUSELLE (Assistant technique et)
- Yannis OLLOMO BEKONO (Gestion du projet)



Outils Utilisés



Code de base

```
// C program to implement an interactive
// snake game
#include <conio.h>
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>

// Macros variable (HEIGHT, WIDTH)
#define HEIGHT 20
#define WIDTH 40

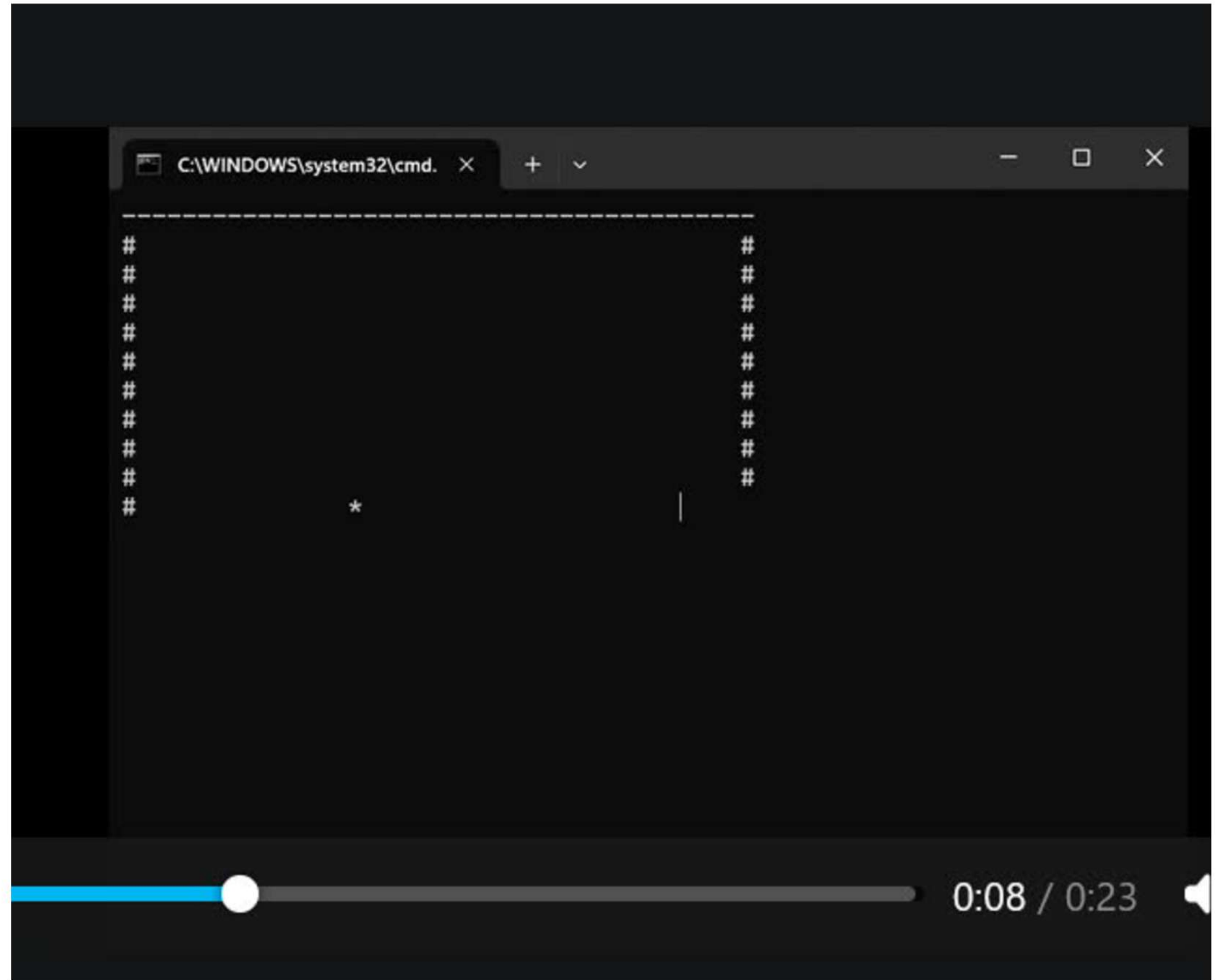
// Array to store the coordinates of snake
// tail (x-axis, y-axis)
int snakeTailX[100], snakeTailY[100];

// Variable to store the length of the
// snake's tail
int snakeTailLen;

// Score and signal flags
int gameover, key, score;

// Coordinates of snake's head and fruit
int x, y, fruitx, fruity;

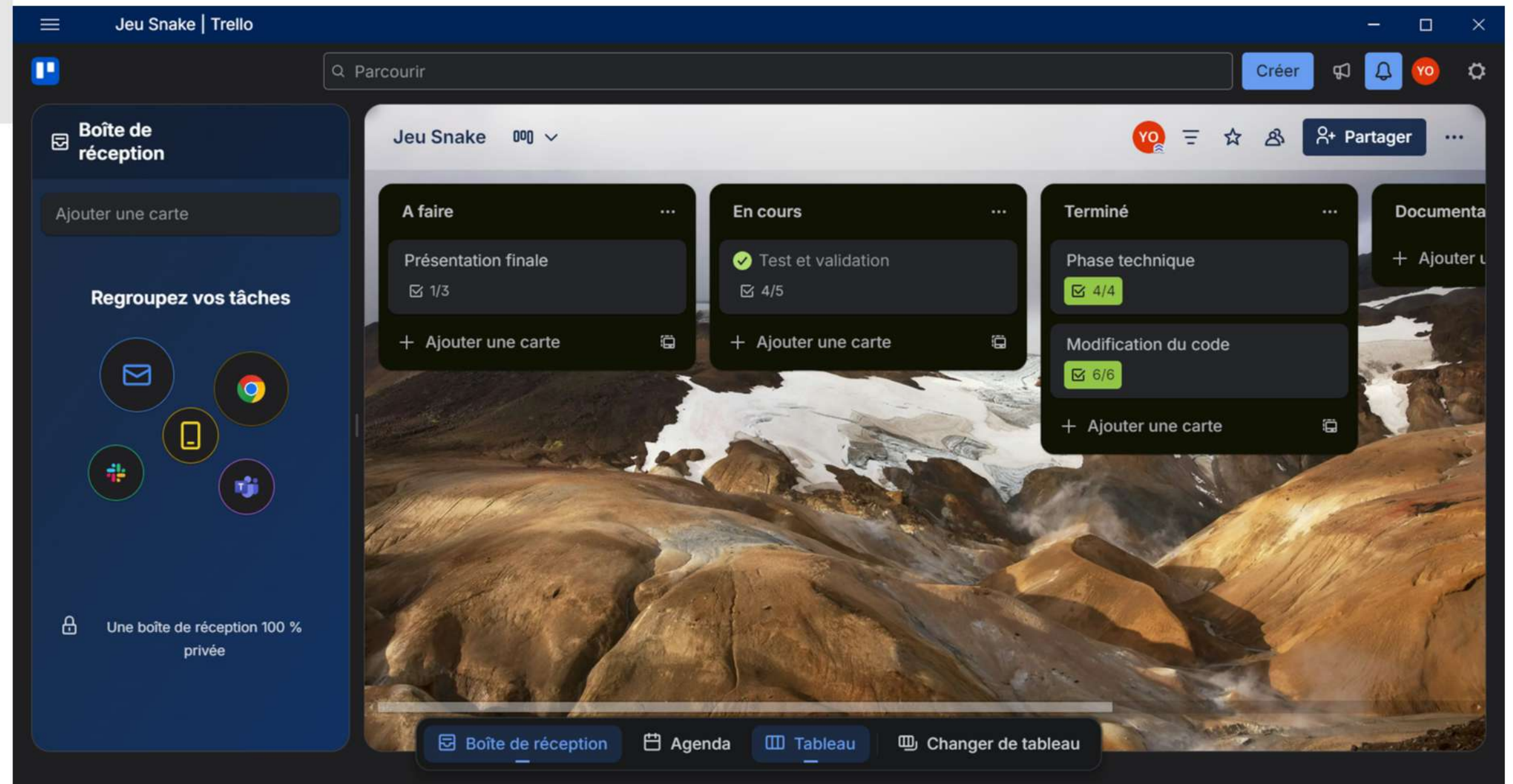
// Function to generate the fruit
// within the boundary
void setup() {
```



Source du code : <https://www.geeksforgeeks.org/c/snake-game-in-c/>

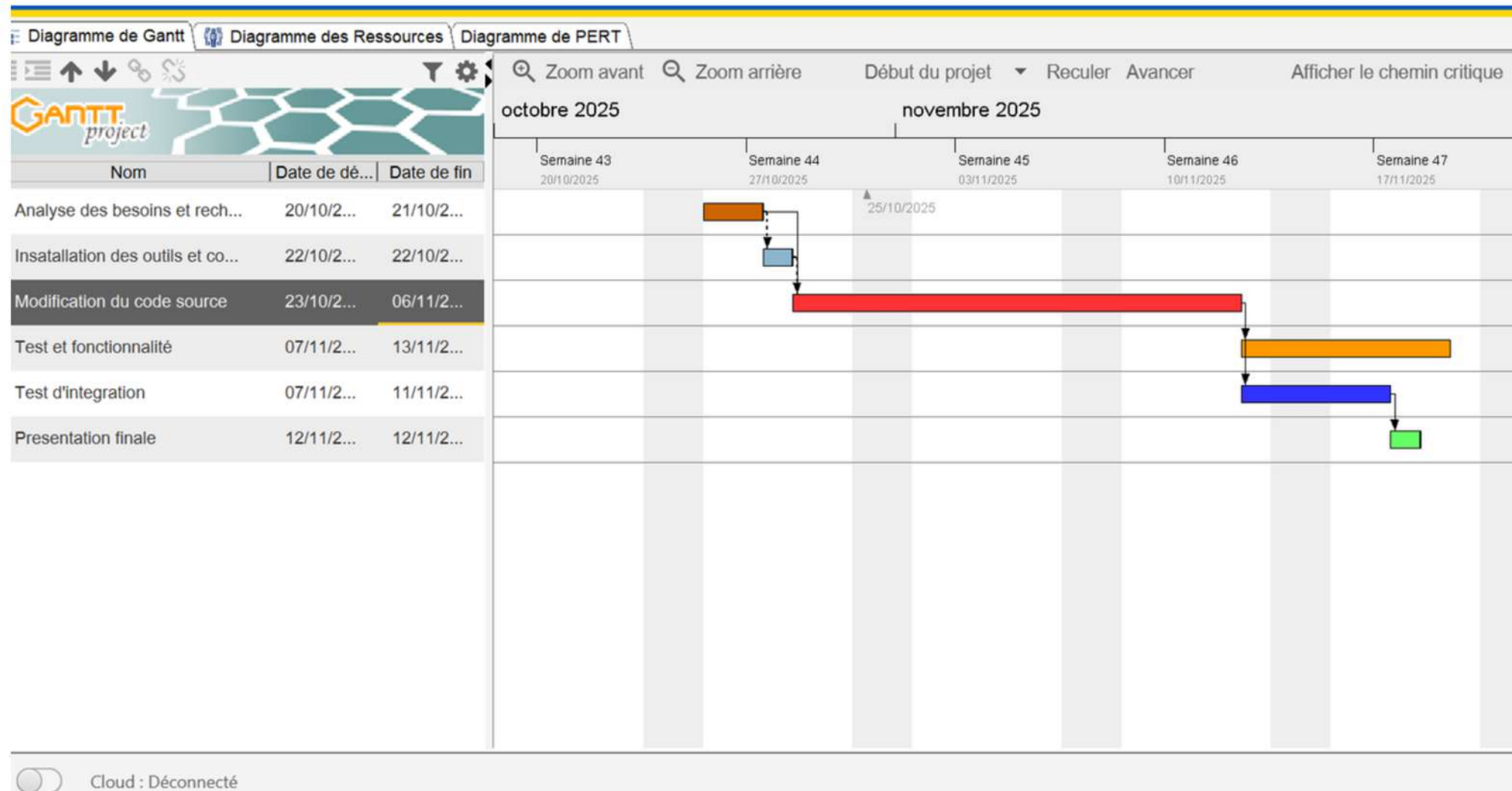
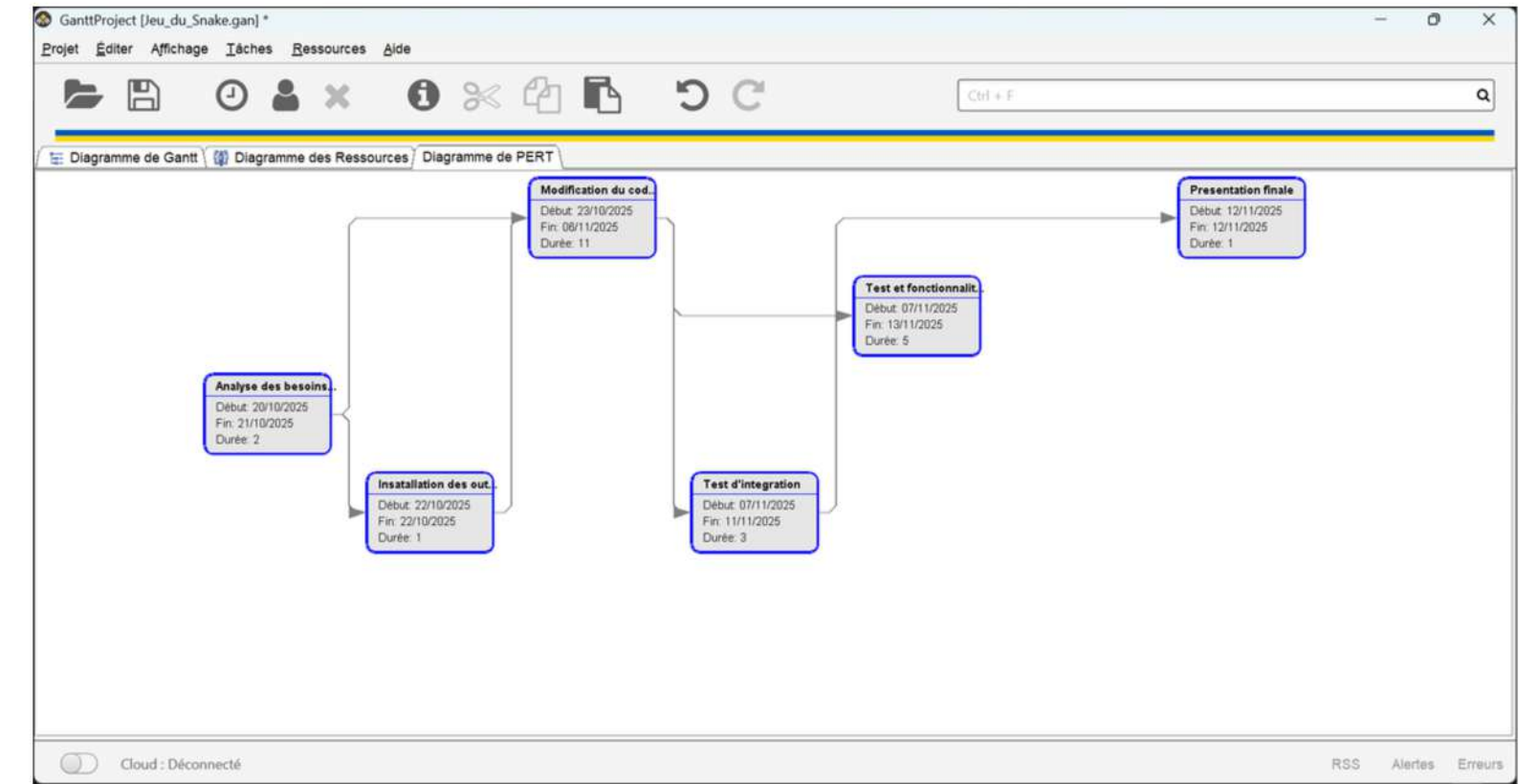
Présentation Trello

Le projet était prévu sur une durée de 2 semaines à partir du moment où nous aurons reçu l'ensemble des éléments nécessaires à la réalisation de la prestation.





Présentation du diagramme de Gantt et Pertt





FONCTIONNALITÉ ET STRUCTURE DU
PROGRAMME

Jeu snake

- Le serpent se déplace dans une grille (40x20)
- Il mange des pommes (*) pour gagner des points
- Il s'allonge après avoir mangé 3 pommes
- Le jeu se termine :
 - s'il touche un mur
 - s'il se mord la queue

- Flèches directionnelles ↑ ↓ ← → ou W / A / S / D
- P → sauvegarder la partie
- L → charger une partie
- X → quitter
- 1 ou 2 → choix de la vitesse (lente / rapide)

Structure du programme

Fonction	Rôle
<code>setup()</code>	Initialise le jeu (serpent + pommes)
<code>draw()</code>	Affiche le plateau de jeu
<code>input()</code>	Gère les touches du clavier
<code>logic()</code>	Gère les collisions et le score
<code>saveGame()</code> / <code>loadGame()</code>	Sauvegarde ou charge une partie

AFFICHER 3 POMMES AU LIEU DE UNE

Modification apporté

```
void setup() {  
  
    // Flag to signal the gameover  
    gameover = 0;  
  
    // Initial coordinates of the snake  
    x = WIDTH / 2;  
    y = HEIGHT / 2;  
  
    // Initial coordinates of the fruit  
    fruitx = rand() % WIDTH;  
    fruity = rand() % HEIGHT;  
    while (fruitx == 0)  
        fruitx = rand() % WIDTH;  
  
    while (fruity == 0)  
        fruity = rand() % HEIGHT;  
  
    // Score initialized  
    score = 0;  
}
```

```
// ** Setup initial avec 3 pommes **  
void setup() {  
    gameover = 0;  
    key = 0;  
    x = WIDTH / 2;  
    y = HEIGHT / 2;  
  
    for (int i = 0; i < 3; i++) {  
        fruitx[i] = rand() % WIDTH;  
        fruity[i] = rand() % HEIGHT;  
        while (fruitx[i] == 0)  
            fruitx[i] = rand() % WIDTH;  
        while (fruity[i] == 0)  
            fruity[i] = rand() % HEIGHT;  
    }  
  
    snakeTailLen = 0;  
    score = 0;  
}
```

AFFICHER LE TERRAIN, LE CORPS DU SNAKE ET LES
POMMES

Modification apporté

```
// Creating the sanke's food with '*'
else if (i == fruity && j == fruitx)
    printf("*");

// Creating snake's body with 'o'
else {
    int prTail = 0;
    for (int k = 0; k < snakeTailLen; k++) {
        if (snakeTailX[k] == j
            && snakeTailY[k] == i) {
            printf("o");
            prTail = 1;
        }
    }
    if (!prTail)
        printf(" ");
}
```

```
else {
    int prTail = 0;
    for (int k = 0; k < snakeTailLen; k++) {
        if (snakeTailX[k] == j && snakeTailY[k]
            == i) {
            printf("o");
            prTail = 1;
        }
    }
    int prFruit = 0;
    for (int f = 0; f < 3; f++) {
        if (fruitx[f] == j && fruity[f] == i) {
            printf("*");
            prFruit = 1;
        }
    }
    if (!prTail && !prFruit)
        printf(" ");
}
```



```

// If snake reaches the fruit
// then update the score
if (x == fruitx && y == fruity) {
    fruitx = rand() % WIDTH;
    fruity = rand() % HEIGHT;
    while (fruitx == 0)
        fruitx = rand() % WIDTH;

    // Generation of new fruit
    while (fruity == 0)
        fruity = rand() % HEIGHT;
    score += 10;
    snakeTailLen++;
}

```

VERIFICATION SI LE SNAKE MANGE LES TROIS POMMES
IL S'AGRANDIT ET GENERER TROIS POMMES AVEC DES
EMPLACEMENTS ALEATOIRE

Modification apporté

```

// ** Vérifier si le serpent mange une pomme parmi les 3 **
for (int i = 0; i < 3; i++) {
    if (x == fruitx[i] && y == fruity[i]) {
        // Mettre cette pomme hors de l'écran pour marquer qu'elle a été mangée
        fruitx[i] = -1;
        fruity[i] = -1;
        score += 10;
    }
}

// ** Si les 3 pommes sont mangées, agrandir serpent et régénérer 3 pommes **
int eatenCount = 0;
for (int i = 0; i < 3; i++) {
    if (fruitx[i] == -1 && fruity[i] == -1)
        eatenCount++;
}

if (eatenCount == 3) {
    snakeTailLen++;
    for (int i = 0; i < 3; i++) {
        fruitx[i] = rand() % WIDTH;
        fruity[i] = rand() % HEIGHT;
        while (fruitx[i] == 0)
            fruitx[i] = rand() % WIDTH;
        while (fruity[i] == 0)
            fruity[i] = rand() % HEIGHT;
    }
}

```


Ajout des fonctions

```
// ** Ajout : fichier sauvegarde **
const char* saveFile = "snake_save.dat";

// ** Fonction pour sauvegarder le jeu **
void saveGame() {
    FILE *f = fopen(saveFile, "wb");
    if (f == NULL) {
        printf("Erreur sauvegarde\n");
        return;
    }
    fwrite(&x, sizeof(int), 1, f);
    fwrite(&y, sizeof(int), 1, f);
    fwrite(fruitx, sizeof(int), 3, f);
    fwrite(fruity, sizeof(int), 3, f);
    fwrite(&snakeTailLen, sizeof(int), 1, f);
    fwrite(snakeTailX, sizeof(int), snakeTailLen, f);
    fwrite(snakeTailY, sizeof(int), snakeTailLen, f);
    fwrite(&score, sizeof(int), 1, f);
    fwrite(&key, sizeof(int), 1, f);
    fwrite(&gameover, sizeof(int), 1, f);
    fclose(f);
    printf("Jeu sauvegardé.\n");
    Sleep(1000);
}
```

Fonctions utilisées

- fopen(), fwrite(), fclose() → fonctions de la bibliothèque <stdio.h> pour écrire dans un fichier

Rôle

- Permet au joueur de sauvegarder sa partie à tout moment avec la touche P.
 -

Ajout des fonctions

```
// ** Fonction pour charger le jeu **
void loadGame() {
    FILE *f = fopen(saveFile, "rb");
    if (f == NULL) {
        printf("Aucune sauvegarde trouvée.\n");
        Sleep(1000);
        return;
    }
    fread(&x, sizeof(int), 1, f);
    fread(&y, sizeof(int), 1, f);
    fread(&fruitx, sizeof(int), 3, f);
    fread(&fruity, sizeof(int), 3, f);
    fread(&snakeTailLen, sizeof(int), 1, f);
    fread(&snakeTailX, sizeof(int), snakeTailLen, f);
    fread(&snakeTailY, sizeof(int), snakeTailLen, f);
    fread(&score, sizeof(int), 1, f);
    fread(&key, sizeof(int), 1, f);
    fread(&gameover, sizeof(int), 1, f);
    fclose(f);
    printf("Jeu chargé.\n");
    Sleep(1000);
}
```

Fonctions utilisées

- fopen(), fread(), fclose() → lecture du fichier binaire

Rôle

- Permet au joueur de sauvegarder sa partie à tout moment avec la touche L.
 -

AJOUTER LA VARIABLE VITESSE

Ajout d'une variable

```
int main() {
    setup();

    // ** Choix de la vitesse **
    int speedChoice;
    printf("Choisissez la vitesse (1 = lente, 2 = rapide) : ");
    scanf("%d", &speedChoice);
    int delay = (speedChoice == 2) ? 50 : 150; // 50 ms = rapide, 150 ms = lente

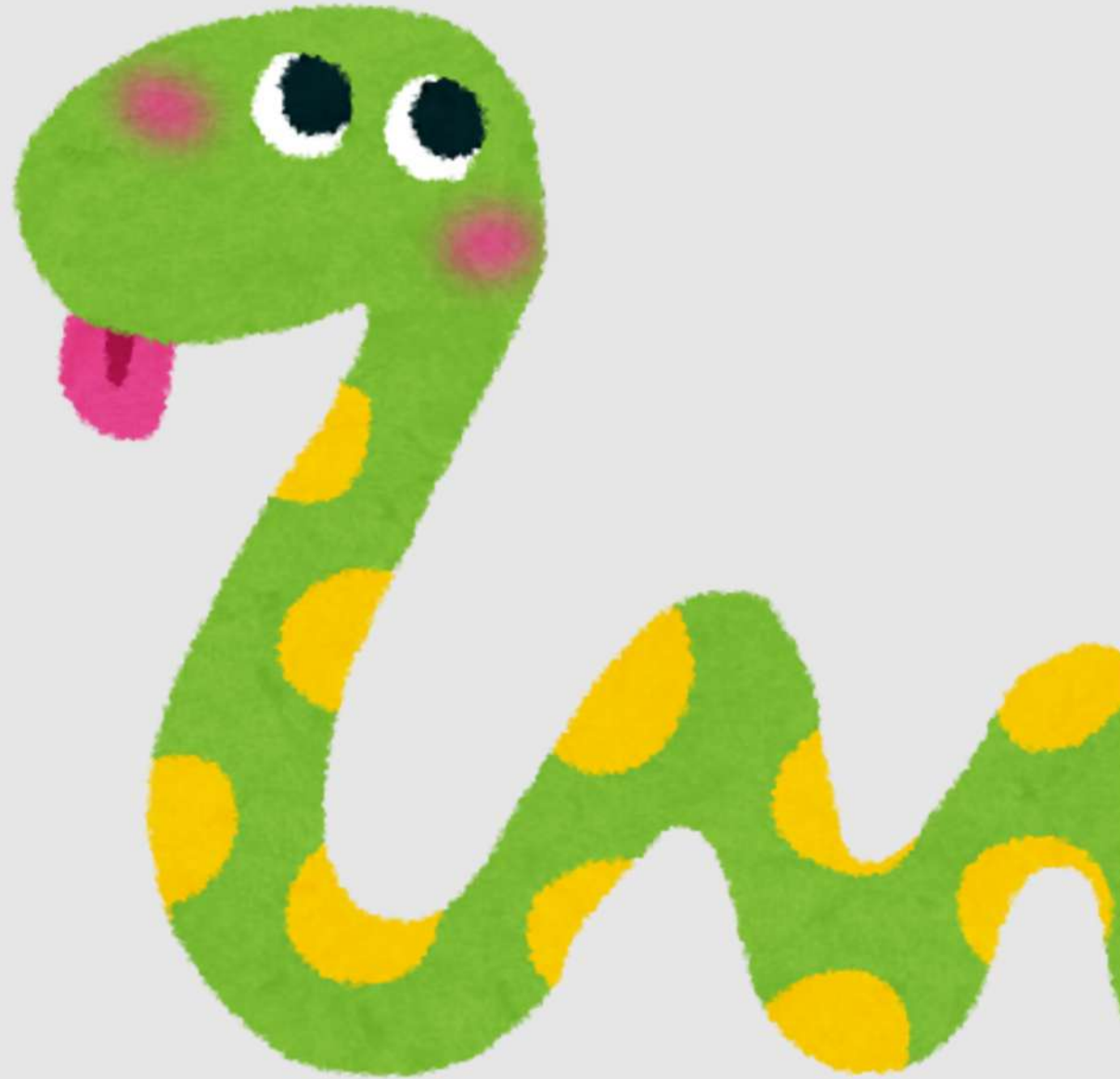
    while (!gameover) {
        draw();
        input();
        logic();
        Sleep(delay); // Utilise la vitesse choisie ici
    }
    printf("Game Over ! Score final : %d\n", score);
    return 0;
}
```

Variables utilisées

- speedChoice → choix du joueur.
- delay → durée de pause entre deux tours du jeu.

Rôle

- Donne la possibilité de jouer plus lentement ou plus vite selon la difficulté souhaitée



Conclusion

Ce projet nous a permis de mettre en pratique nos connaissances en langage C à travers la création d'un jeu interactif et structuré.

Nous avons appris à mieux gérer les tableaux, les boucles, les fonctions et la gestion des fichiers pour sauvegarder et charger une partie.

La réalisation du Snake nous aussi aidé à comprendre l'importance de la logique de programmation et de l'organisation du code.